

Projet_SE_1 – Rapport TP1 Nachos

1 Introduction / Bilan

Dans ce TP, nous avons mis en place de nouvelles fonctions de communication entre programmes utilisateurs et le noyau de Nachos. Ces fonctions comprennent :

- `GetChar` : lecture d'un caractère depuis le clavier.
- `PutString` / `copyStringFromMachine` / `copyStringToMachine` : manipulation sécurisée des chaînes entre l'espace utilisateur et le noyau.
- `GetString` : lecture d'une chaîne de caractères terminée par `\n` ou `\0`.
- `PutInt` / `GetInt` : conversion entre entiers et chaînes de caractères, basée sur `snprintf` et `sscanf`.

Nous avons implémenté `copyStringToMachine` pour assurer la copie correcte des données depuis la mémoire noyau vers la mémoire utilisateur. Les appels système ont été testés avec succès grâce aux programmes présents dans le répertoire test.

2 Points sensibles

La principale difficulté a été la gestion de l'interaction entre l'espace noyau et l'espace utilisateur.

2.1 Gestion des chaînes

- `copyStringFromMachine` : recopie d'une chaîne depuis la mémoire utilisateur vers un buffer noyau. Il a fallut :
 - faire attention à la taille de la chaîne de caractère (buffer) ;
 - garantir la sécurité de la mémoire et l'intégrité de l'information récupérée.
- `copyStringToMachine` : recopie inverse d'une chaîne depuis le noyau vers la mémoire utilisateur. Il a fallut :
 - garantir l'absence de débordement en respectant la taille maximale du buffer utilisateur ;
 - ajouter systématiquement un caractère de terminaison `\0` ;
 - éviter les fuites mémoire en libérant le buffer temporaire utilisé dans le noyau.

2.2 Lecture/écriture sécurisée

L'argument `int *value` de `ReadMem` : on ne peut pas lui passer directement un pointeur sur le buffer `to`, car `ReadMem` écrit un entier (4 octets), alors que `to` est un `char` (1 octet). Passer le pointeur directement pourrait écraser la mémoire et provoquer des erreurs.

2.3 Allocation des buffers

- Il n'est pas raisonnable d'allouer un buffer de la même taille que la chaîne MIPS, car l'utilisateur pourrait fournir une chaîne extrêmement longue.
- Sans limitation (`MAX_STRING_SIZE`), cela pourrait saturer la mémoire du noyau ou provoquer des erreurs si la chaîne dépasse plusieurs gigaoctets.

- Avec un buffer fixe, on garantit que la mémoire consommée par le noyau reste raisonnable et que la chaîne est fractionnée si nécessaire.

3 Gestion de la fin de programme

Retirer `Halt()` à la fin du programme test `putchar.c` provoque une exception lors du syscall 2, correspondant à `SC_Exit`. Cette erreur survient car le code utilisateur envoie systématiquement le syscall 2 à la fin de `main()`, mais le noyau ne gère pas encore cette exception.

3.1 Solution

Implémentation d'un traitant pour `SC_Exit` dans `exception.cc`, basé sur `Halt()` :

- lecture du code de retour de `main` via `machine->ReadRegister(4)` ;
- terminaison propre du programme avec `interrupt->Powerdown()`.

Ainsi, les programmes utilisateur n'ont plus besoin d'appeler explicitement `Halt()`, et le code de retour de `main()` peut être récupéré correctement dans le débogueur.

4 Tests réalisés

Nous avons conçu plusieurs programmes simples pour valider nos appels système, soit un pour chaque fonction implémentée dans `consoledriver.cc`

4.1 Observations

- Les tests ont été effectués sur des cas simples (chaînes courtes, entiers standards) et des cas limites (chaînes proches de la taille maximale, lectures consécutives).
- Les sorties observées correspondent aux attentes.
- La gestion sécurisée des buffers (`MAX_STRING_SIZE`, ajout de `\0`) empêche tout dépassement mémoire (aucun problème avec `-fsanitize=address`).

5 Conclusion

La principale complexité du TP réside dans la sécurisation des échanges entre mémoire utilisateur et noyau. La limitation de taille des buffers et l'utilisation des fonctions `copyStringFromMachine` et `copyStringToMachine` permettent d'éviter les débordements et les erreurs mémoire. Les tests montrent que les appels système sont fonctionnels et respectent les comportements attendus.